

# Panorama de Harness de LLM

As ferramentas que transformam modelos em agentes de código — Claude Code, Cowork, OpenClaw, OpenCode e Codex.

 Panorama executivo — 2026

## O que é um harness de LLM?

Um harness é a camada de software que coloca um LLM dentro de um loop agente — orquestrando ferramentas, contexto, sistema de arquivos, terminal e permissões para que o modelo execute trabalho real, não apenas gere texto. Em resumo: **LLM + loop + ferramentas = agente útil** (prompt → ação → observação → próximo passo).

|             |                                             |              |                                       |
|-------------|---------------------------------------------|--------------|---------------------------------------|
| LOOP AGENTE | Plano → tool call → resultado<br>→ reflexão | FERRAMENTAS  | Edição de arquivos, bash,<br>web, MCP |
| CONTEXTO    | Memória, arquivos abertos,<br>histórico     | SALVAGUARDAS | Permissões, sandbox,<br>confirmações  |

### Por que o harness virou o produto

|                      |                                           |                      |                                                      |
|----------------------|-------------------------------------------|----------------------|------------------------------------------------------|
| 80% DO VALOR PRÁTICO | de um LLM vem do harness<br>ao redor dele | 10X DE PRODUTIVIDADE | de diferença entre harnesses<br>bons e ruins         |
| +5 FERRAMENTAS       | amplamente usadas em<br>produção em 2026  | BASE                 | Estimativas qualitativas,<br>baseadas em uso prático |

- **Modelos viraram commodity** — a diferenciação está na orquestração: como o agente pensa, edita e verifica.
- **Trabalho real exige guardas** — permissões, sandbox e revisão humana definem o que pode ir para produção.
- **Ecossistema MCP/plugins** — tools, skills e conectores agora são portáteis entre harnesses.

### Mapa: quatro harnesses em destaque

- **Claude Code · Cowork** (Anthropic) — CLI agente + modo desktop para não-devs. Plugins, skills, MCP, hooks.
- **OpenClaw** (open-source, comunidade) — harness comunitário, compatível com Claude e outros modelos.
- **OpenCode** (open-source, comunidade) — agente de codificação em terminal, multi-modelo, foco em TUI.
- **Codex** (OpenAI) — CLI agente da OpenAI, integrado ao ecossistema GPT / Codex.

## As quatro ferramentas, em poucas linhas

- **Claude Code & Cowork** (Anthropic) — CLI agente de codificação: roda no terminal, edita arquivos, executa comandos, lê o repositório inteiro; Cowork leva isso ao desktop para não-devs. Extensível via plugins, skills e MCP/hooks, com permissões e confirmações para ações sensíveis. Forte para refatoração multi-arquivo, agentes longos e workflows com skills.
- **OpenClaw** (open-source) — harness comunitário compatível com a família Claude e outros provedores, com foco em transparência, customização e bring-your-own-key (BYOK). Multi-modelo, hackável, sem lock-in — código aberto com auditoria total do loop e das tools. Projeto emergente: vale checar o repositório oficial antes de adotar; documentação ainda leve.
- **OpenCode** (open-source) — agente de codificação no terminal com experiência TUI rica, provider-agnóstico (Anthropic, OpenAI, Groq, modelos locais). Tooling profundo (edição, shell, busca, git) direto no TUI, sessões compartilháveis e resposta rápida por rodar nativo em terminal.
- **Codex** (OpenAI) — CLI agente de codificação integrado ao ecossistema GPT, disponível também via Web/IDE. Modos de operação graduais: Suggest (propõe e pede confirmação), Auto Edit (edita e pergunta antes de rodar comandos) e Full Auto (executa tudo em sandbox, para tarefas longas).

### Como elas se posicionam

- **Origem** — Anthropic (Claude Code/Cowork) · comunidade (OpenClaw, OpenCode) · OpenAI (Codex).
- **Modelo padrão** — Claude/Sonnet-Opus-Haiku · multi-provider BYOK (OpenClaw) · multi-provider (OpenCode) · GPT/Codex (OpenAI).

- **Interface** — CLI + desktop Cowork · CLI (OpenClaw) · TUI rica (OpenCode) · CLI + Web/IDE (Codex).
- **Licenciamento** — proprietário com tier free (Claude Code) · open-source (OpenClaw, OpenCode) · proprietário (Codex).
- **Público-alvo** — devs e não-devs via Cowork · devs/pesquisadores (OpenClaw) · devs de terminal (OpenCode) · devs do ecossistema OpenAI (Codex).

## Qual harness escolher

- **Claude Code / Cowork** — quando você quer o melhor agente Claude com plano integrado de plugins + skills, ou quer trazer não-devs para o fluxo.
- **OpenClaw** — quando precisa de transparência total, customizar o loop, ou rodar local com chaves próprias — aceitando curva técnica maior.
- **OpenCode** — quando o terminal é o seu lar, você quer uma TUI rica e liberdade para misturar provedores sem amarras de uma stack só.
- **Codex** — quando já está no ecossistema OpenAI e quer um agente CLI maduro, com modos de aprovação granulares e companion IDE/Web.

## Mensagens-chave: o harness virou o produto

- 01** Modelos são camada, agentes são produto — capacidade bruta sem orquestração não entrega trabalho; o loop, as tools e os guard-rails fazem a diferença prática.
- 02** Quatro filosofias, um mesmo objetivo — Claude Code/Cowork (integrado), OpenClaw (hackável), OpenCode (TUI portátil), Codex (OpenAI nativo). Cada um cobre um perfil.
- 03** Avalie pelo encaixe, não pelo hype — tipo de tarefa, perfil do usuário, modelo preferido e nível de controle são as variáveis que devem guiar a escolha.

*Quase ninguém usa um único harness — a combinação certa depende da tarefa.*

Fonte: material do curso — slide deck Harness\_de\_LLM\_Panorama.pptx (IntroducaoCurso/). Comparativo qualitativo; capacidades específicas evoluem rapidamente — valide sempre com a documentação oficial de cada ferramenta.